International Journal of Computing and Digital Systems
2025, VOL. 18, NO. 1, 1-15

http://dx.doi.org/10.12785/ijcds/1571149246

# Comparative Analysis of LSTM and CNN Architectures for Prediction Enhancement: Case of Moroccan Stock Market Return

**Ayman Mazzouz**[1] and **Abdelali Elmoufidi**[2]

[1]*Data4earth Laboratory, Faculty of SciencesandTechnics, Sultan Moulay Slimane University, Beni Mellal, Morocco*
[2]*Prospective Research Laboratory in Economics and Management, National School of Commerce and Management, Hassan 2 University, Casablanca, Morocco*

*Received 13 November 2024, Revised 05 July 2025, Accepted 15 August 2025*

**Abstract:** It is an important activity performed by academics, investors, and financial institutions to forecast financial time series and predict the movement of the stock market. The financial markets of today are deeply interconnected, making them sensitive not only to economic indicators but also to political, social, and even environmental factors. In such a volatile context, the ability to anticipate market direction with precision becomes a key advantage for decision-makers seeking to achieve consistent and competitive performance. Prediction of the right market movement of stocks is therefore extremely important to support investment strategies and risk management policies. The current study compares two deep learning models for the prediction of the returns on the Moroccan All Shares Index (MASI): Long Short-Term Memory (LSTM) and one-dimensional Convolutional Neural Networks (1D-CNN). Both models were trained and tested on historical MASI data, with 80% of the dataset used for training and 20% for testing. EarlyStopping was employed to avoid overfitting and ensure optimal learning. Their performance was assessed using a variety of evaluation metrics and supported by multiple visualizations, including trend tracking, learning curves, and residual error analysis. This thorough approach allowed for a fair and robust comparison, highlighting the differences in predictive accuracy, stability, and generalization ability between the two models. The results indicate that the LSTM model outperforms the CNN model in terms of predictive accuracy, stability, and alignment with actual market trends. These findings reinforce the strengths of LSTM in financial time series forecasting, suggesting it as a more reliable and effective tool for MASI prediction and potentially other similar financial indices, particularly in emerging markets where data patterns are complex and highly dynamic.

**Keywords:** MASI, LSTM, CNN, Stock prediction

## 1. INTRODUCTION

Trading and forecasting financial stocks demand a difficult and hard task for market members and participants. Prediction financial time-series can be very rigorous because they are impacted by a large number of factors, as we know the time-series is a group of data points [1] measured, collected, or recorded sequentially over time indexed by timestamps that serve as digital fingerprint or markers allow us understanding the temporal purposes context of data.Neural networks, on the other hand, have become extremely prominent in the field of stock market prediction. Convolutional neural networks (CNN), recurrent neural networks (RNN), long short-term memory (LSTM), and gated recurrent units (GRU) are just a few of the neural network types that have emerged as effective tools in this field. RNNs handle information in sequential steps, in contrast to standard neural networks, which treat inputs and outputs as separate entities [2]. When developing forecasting systems, this sequential processing is very useful since it enables models to efficiently learn from historical data.

Because Robj. Hyndman, who is based on statistical methods for time series, has multiple prominent books and papers on forecasting methods, researchers who want to build robust forecasting systems frequently look for the most informative input features [3]. Significant lags from time series models, such as the Box Jenkins model, RNNs are excellent at carrying out consistent tasks on every element in a sequence and using the results for further calculations [4]. Beyond market prediction, RNN is used in a variety of applications, face detection, and brainwave signal identification. Crowd flow forecasting, clinical medical

*E-mail address: mazzouzayman@gmail.com, elmoufidi14@gmail.com*

applications, weather forecasting, rainfall prediction, wind speed forecasting, electrical load prediction, human behavior forecasting, extreme event forecasting, cargo forecasting, and traffic flow forecasting are just a few of the time-series forecasting problems that can be resolved by adapting the methodology [5], [6]. Through their trading positions, market participants hope to generate a possible return. However, a number of factors influence stock prices [7], including the inflation rate, political changes, economic conditions, global health crises or pandemics [8], and traders' technical skills. In order to accurately forecast changes and trends in stock prices, they search for high-accuracy models that resist overfitting.

Initially, financial institutions, in the prediction of security prices, used statistical models such as ARIMA (Autoregressive integrated moving average, and GARCH (generalized autoregressive conditional heteroskedasticity) that uses the residuals (the differences between the actual stock prices and the prices predicted ) or the combination between statistical models to predict the future price and expected volatility.

The implementation of machine learning algorithms has enabled investors to generate substantial profit. In fact, statistical models cannot handle all the complexity of the financial markets, and also the non-linearity of the time series. Recently, RNNs architecture family has proved to be the most suitable for the prediction process.

In this paper, we plan to compare and evaluate the performance of LSTM, CNN used for forecasting stock market indices, the major contributions in this paper are as follows.

- We compare and implement two deep learning models (LSTM and CNN) with one common financial time series data.
- We analyze and contrast their forecasting abilities based on different criteria.
- We provide empirical examination of the strengths and limitations of each model in stock market forecasting.

The remainder of the paper is organized as follows: **Section 2 includes a brief Related work; Section 3 describes the adopted methodology; Section 4 presents the experiments settings; Section 5 provides results. Finally, Sections 6 draws the main conclusion.**

## 2. Related work

According to A. Rjumohan, [9] investors and financial institutions use online platforms to conduct economic transactions involving the purchase and sale of shares, bonds and other financial instruments. These platforms, called stock markets, serve as a gathering place for individuals and institutions interested in generating substantial return based on the anticipated market movement. However, predicting stock market prices is a complex endeavour [10], [11]. Throughout the years, various theories have emerged with the aim of elucidating the workings of stock markets or determining whether they can be outperformed. One of the most renowned and contentious theories, as proposed by Fama [12] in 1970, is the Efficient Market Hypothesis (EMH). This hypothesis asserts that, at any given moment, a stock's market price encapsulates all available information concerning that stock. In simpler terms, a stock is accurately valued until some alteration in its circumstances transpires. In Morocco, the platform where the investors, individuals and financial institutions forecasting called Casablanca stock exchange, founded in 1929 [13] works as a public limited company with private capital, it operates under controlling of the Moroccan government [14], that responsible for the promotion of the Moroccan stock market exchange, also it publishing the market information, statistics, and annual report etc. . . [15].

### *a) Lstm*

In this article we applied the RNN that introduced by Jeff Elman and the concept of this technology (RNN) [16] is to address the challenges associated with processing sequential data as primary purpose, such as text, voice, and video. This type of data exhibits a sequential relationship between its individual samples, where each sample is linked to the one that precedes it. For instance, in textual data, a word is connected to the word immediately preceding it. In meteorological data, the temperature of one day is influenced by the temperatures of the preceding days. A collection of observations is defined as a sequence, from which multiple sequences can be derived[17]. Hochreiter and Schmidhuber [18] presented LSTM, one of the top RNN models for time series and prediction stocks, in 1997. The neuron architecture of LSTMs differs significantly from that of conventional Recurrent Neural Networks. In contrast to conventional RNNs and each neuron in an LSTM acts as a "memory cell," in contrast to typical neural networks where neurons are indistinguishable. These memory cells are essential for relating previously learned material to the current job. The input gate, forget gate, and output gate are the three fundamental gates included in LSTM networks [19]. Usually represented by the letter "f," the activation function is a sigmoid function with a range of 0 to 1.

Ricchiuti and Sperlí (2025) [20] proposed an LSTM-based predictive framework for forecasting the stock market that integrates financial time series, technical analysis indicators, news sentiment, and seasonality. Its core is an LSTM model, chosen due to its ability to process long-term and nonlinear dependency in sequential financial data. By intensive comparative analysis, LSTM outperformed GRU, Bi-LSTM, Bi-GRU, and several stacked configurations with respect to MAE, RMSE, and MAPE. The study confirmed that predicting the daily returns rather than the closing prices is more precise using LSTM. To assist with invest-

ment decisions, a heuristic stock selection algorithm was proposed, ordering stocks according to prediction confidence. The model was tested on 417 NASDAQ stocks and 67 cryptocurrencies and returned a total return of 41.21% and 39.38%, respectively. The results above confirm the strong predictive power of LSTM when enhanced with diverse financial and contextual features

Qin et al. (2024) [21] proposed an EWT-LSTM hybrid model to improve the accuracy of financial time series forecasting. The process first decomposes raw time series into a number of inherent mode functions (IMFs) through EWT so that the model can deconstruct the model into various components based on frequencies. Each IMF is then fed into an individual LSTM network to extract temporal patterns at different levels of resolution, and the final prediction is obtained by aggregating the output of all LSTM models. Six global stock indexes were compared using comparative tests, NASDAQ, SP 500, and FTSE, based on RMSE and MAE as assessment metrics. The tests confirmed that the EWT-LSTM model significantly outperformed other common models such as ARIMA, LSTM, and CNN, particularly in detecting complex non-stationarity behavior in the financial market. This study confirms that the use of EWT as a preprocessing step enhances the performance of LSTM for forecasting financial time series with multi-scale features.

Agarwal and Sharma (2025) [22] introduced a hybrid framework that combined Singular Value Decomposition-based Mode Decomposition (SVMD) and Long Short-Term Memory (LSTM) for better time-series forecasting. The SVMD technique is employed as a preprocessing procedure to decompose the raw time series into a finite set of known-frequency components, removing noise and disentangling patterns of interest. Individual decomposed modes are modeled individually by LSTM networks, and the ultimate prediction is created by aggregating all forecasted components. The model was experimented with financial and weather-related data, including stock indices, and compared with baseline models such as isolated LSTM, GRU, SVR, and CNN. Results showed that the SVMD-LSTM model outperformed the benchmarks on several fronts, particularly on RMSE and MAE. The findings determine that SVMD improves the forecasting performance of LSTM by enhancing its ability to learn from denoised and structurally regularized input sequences.

*b) cnn*

Also we applied the CNN technology, particularly a 1D-CNN, That mentioned by Yann LeCun, defined the technology of CNN [23] as a means to handle spatial and time series data types that include audios, sensor data and market trends. Such data is one-dimensional, meaning that each sample has a sequential relationship with the previous or the next one. For example, audio data has samples depending on those preceding it and following it to make up uninterrupted sound waves. While in financial time series prices at one point are caused by prices at some previous points. A sequence consists of consecutive points from which many other similar sequences could be derived. 1D CNN is one of the best models used in time-series analysis and pattern recognition by CNNs. This architecture is unique, compared to CNNs that were built for images. Different from the usual CNN networks that have filters scanning across a 2-dimensional space, each filter of a 1D-CNN runs only along one dimension to extract temporal or sequential features. These types of nets are more applicable for item extraction from fixed-length segments of the entire dataset where convolution layers capture local dependencies while pooling layers reduces dimensionality enhancing the learning process. Additionally, these activation functions also allow non-linearity through the ReLU function which enhances complex pattern recognition when dealing with serial data

Zhang et al. (2024) [24] introduced an adaptive multimodal learning model of financial market price forecasting integrating textual, numerical, and visual data through a Convolutional Neural Network (CNN) backbone. The novel architecture integrates sentiment signals from financial news headlines, technical indicator signals, and candlestick chart image signals processed in parallel through separate CNN pipelines. Each CNN module learns the right features from the respective modality, and predictions are fused using a dense attention mechanism prior to the final prediction. The model was evaluated on SP 500 stock price data and achieved improved performance compared to the baseline CNN, LSTM, and BERT-based models. The results confirm CNN's ability to effectively extract localized time series and visual patterns, particularly when utilized to process multimodal inputs for financial forecast applications.

Singh et al. (2023) [25] proposed the use of a CNN-based architecture for stock price prediction in the stock market, for local temporal pattern extraction from historical time series data. The model employs 1D convolution layers and max-pooling followed by dense layers in order to automatically learn high-level features without feature engineering. The experiment was intended to predict closing prices of major Indian stock indices, and the performance was evaluated in terms of RMSE and MAE. Comparative experiments with ARIMA, SVM, and LSTM confirmed that the CNN model possessed superior prediction error, particularly under short-term forecasting conditions. The authors emphasized CNN's computational efficiency and ability to generalize over structured financial inputs, especially when the time window is fine-tuned. The outcome confirms CNN's applicability in real-time financial forecasting where speedy training and inference are critical.

Khodaei and Mohammadi [26] proposed a CNN-NLP hybrid approach to determine the investor sentiment on Persian-language financial reviews of stock market symbols. The approach uses natural language processing processes along with a Convolutional Neural Network

to mark textual comments as positive or negative. Preprocessed Persian text was converted into vector forms based on ParsBERT embeddings and input into a CNN model that efficiently extracted semantic and spatial features from the sequences. Validated on a sample of 2,990 labeled feedback samples from financial forums and websites, the CNN model achieved more than 90% accuracy, outperforming traditional machine learning classifiers such as SVM and Naïve Bayes. The study demonstrated CNN's effectiveness in sentiment analysis for finance, especially low-resource languages, and in generalizing well on unstructured financial text data.

*c) CNN-LSTM as hybrid model*

In fact, various research works have already established evidence that financial time series forecasting using CNN and LSTM architecture integration works well. Lu et al. (2020) [27] proposed a CNN-LSTM model for stock price prediction; it makes use of the capability of CNN in feature extraction and LSTMs for learning sequences. It yielded a better performance in the forecast of the Shanghai Composite Index, with low rates of prediction error in comparison with traditional methods, such as RNN and independent LSTM methods. Based on this approach, an extended version is done by Wu et al. (2023)[28], including more leading indicators, such as options data and future impacts, in addition to historical stock prices. Their CNN-LSTM model was applied to both US and Taiwanese stock markets, achieving higher accuracy by integrating spatial and temporal features.

The application of deep learning models goes beyond traditional stock markets into the domain of cryptocurrencies. Zhang [29] applied the CNN-LSTM model to predict the Bitcoin price by tuning its performance on daily data from 2017 to 2023. It highly accurately predicted the price, although the results were described as improvable by adding sentiment analysis and more external data. Then there is the use of the CNN-LSTM model in crypto market trend prediction by Khattak et al. [30]. However, their approach was rather more profitable, especially for the multi-class classification, which captures the strength of the trend.

There have been other hybrid architectures developed in relation to financial forecasting. Yadav and Vishwakarma [31] integrated CNN, BiLSTM, and Attention Mechanism combined with blockchain technology in developing a stock price predictor. The use of high-frequency intraday data resulted in their model having very low error rates and becoming well-suited for high-frequency trading. Similarly, Li et al. (2023) [32] constructed a CNel to predict copper prices while incorporating various influential factors such as the cost of energy and global economic indicators. The study precisely points out that the model can catch all the spatial and temporal patterns to always remain superior in terms of high accuracy in forecasts compared to traditional models.

Further, Dong and Zhou [33] proposed a new el that combined the CEEMDAN, ARIMA, and CNN-LSTM for gold futures price prediction. They decomposed the time series into intrinsic mode functions and processed the trend components by CNN-LSTM; they obtained lower forecasting errors than when the methods were applied in isolation.

Comparative studies further validated the superiority of the deep learning models. Extensive analyses have been conducted by Ghosh et al. (2024) [34] on various neural network models comprising MLP, RNN, LSTM, and CNN for stock price prediction on NSE and NYSE. Their consistent results show that hybrid models, more precisely a combination of CNN with LSTM, outperform traditional neural networks in terms of capturing nonlinearities inherent in financial time series.

The former studies have already shown a great promise in financial market forecasting. These models leverage the power of CNN models to extract spatial features and LSTMs, which are good at capturing temporal dependencies. This would make them befittingly suitable for the complex patterns inherent in financial time series data, thus pitting our curiosity in comparing both models' performances in forecasting the Moroccan stock market indicator. However, it is apparent that LSTM models perform particularly well in detecting long-term dependencies in sequential finance data, especially when enhanced by preprocessing techniques like EWT and SVMD. Hybrid models consisting of these two have outperformed other statistical models in terms of accuracy consistently. Similarly, CNN-based models are extremely efficient in detecting local temporal patterns and performed adequately in short-term predictions, sentiment analysis, and multimodal input processing. Besides, CNN-LSTM hybrid models leverage the strength of both models—CNN for spatial feature extraction and LSTM for temporal learning—providing improved predictive outcomes in other markets like equities and cryptocurrencies.

Also, despite these advances, the majority of the past research is plagued with problems such as overfitting due to a lack of regularization, poor comparison across varying horizons, and little relevance to new or untapped markets such as the Moroccan stock market. Furthermore, while hybrid models have been widely applied on established indices like the S&P 500 or Shanghai Composite, few have tested their performance in North African markets where data quality, volatility, and investor behavior vary significantly.

Drawing from this background, our study offers a focused comparative analysis between stand-alone CNN and LSTM models for forecasting the Moroccan MASI index. Contrary to existing literature that emphasizes global or hybrid-market approaches, we aim to evaluate the individual strength of these frameworks in a local environment, underscoring their generalizability and resilience in under-

explored financial landscapes. This will allow us to ascertain which model architecture works better when applied in a developing economy with unique macroeconomic and temporal dynamics.

## 3. Methodology

### A. Method

In our research, we have deployed lstm and CNN models specifically to forecast the movements of the Moroccan All Shares Index (MASI). The models process daily closing prices of the MASI, treating them as a time series. To evaluate the LSTM and CNN predictive capabilities, The available data was divided into two sections: the first 80% of the dataset was used to train the models, while the second 20% was used as the test set. A thorough evaluation of the model's performance is made possible by this segment. Additionally, to evaluate the models' efficacy and efficiency in detail. This method offers a more comprehensive knowledge of the model's forecasting ability in addition to testing its adaptability to various market conditions. In our training and testing analysis, we also included measures like Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). These metrics are essential for measuring the model's precision and dependability in forecasting the trends of the MASI, providing a clear understanding of the overall performance of the CNN and LSTM models.

### B. Data

The first stage of this study is download the historical stock market data from Investing platform. It takes into consideration MASI and use it to perform predictive modeling. Such a dataset should be retrieved in CSV format for compatibility with most data analytical tools. The data are organized with different columns for the analysis of separate variables, standardized by scaling the values in each column to the same range so that each feature is comparable. Cleaning of the data involves removal of values assessed as 'NaN' and imputation of missing values to prevent loss of information. This process also integrates open data related files put together into one data set for comprehensive analysis. Finally, the preprocessed data were then divided into two splits: 80% for training and 20% for testing. The former develops predictive models to learn the underlying patterns of the data, while the latter provides an unbiased evaluation that will estimate the performance of the model on unseen data.

TABLE I. Descriptive statistics of MASI indicator

| Index | Study period | Trading days |
|---|---|---|
| MASI | 2019/01/02 – 2023/09/08 | 1171 |

Feature Selection and Neural Network Training:
During this stage, specific attributes that was input into the neural network are identified, such as the date and closing price. These features is then introduced to the neural network for training, aiming to enable the model to make predictions.
The LSTM model structure includes two LSTM layers and a dense layer. The CNN model structure includes one Convolutional layer, one pooling layer, one flattening, layer and two dense layer The both models structure applied:

- A Sequential input method
- The 'adam' algorithm for optimization.

## 4. Experiments settings

The Fig1 presents a dataset of MASI using stacked line graphs. The blue line depicts the daily closing prices of the MASI over 1171 days. The figure forms a pivotal component of a deep learning investigation into future stock market trend prediction, potentially employing an LSTM network and 1D-CNN.

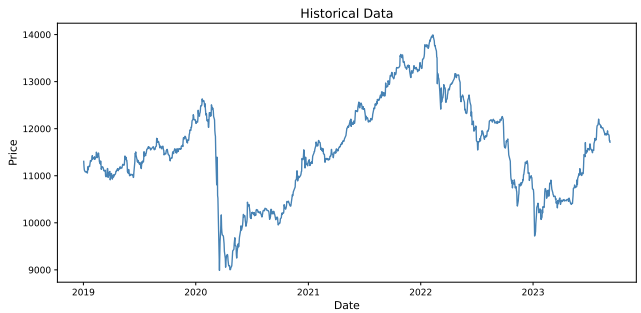


Figure 1. Time Series of Daily Closing Prices for the MASI Index of three different periods

Data Collection: Our study begins with gathering historical stock prices, building the dataset that reflects the MASI's past performance over a chosen period. Imagine this as meticulously compiling a financial diary, charting the index's ups and downs.

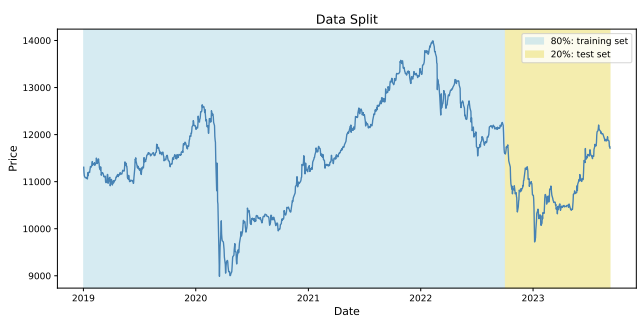


Figure 2. Data split

Dataset Split: Next, we split this diary into two chapters: training and testing sets. Similar to studying for an exam, the larger training set (around 80%) allows the LSTM and CNN models to learn and grasp the market's intricate patterns. The smaller testing set (about 20%) is like the final

exam, where the model applies its acquired knowledge to predict unseen data.

the algorithm 1 and 2 explain how we create the data set and how we prepare it for models training

**Algorithm 1** Create Dataset for Time Series Prediction

**Require:** Data array `data`, time step `time_step`
**Ensure:** Arrays `dataX` and `dataY`
Initialize empty lists `dataX` and `dataY`
**for** each index $i$ from 0 to $len(data) - time_step - 1$ **do**
  Set $a \leftarrow data[i : (i + time_step), 0]$
  Append $a$ to `dataX`
  Append $data[i + time_step, 0]$ to `dataY`
**end for**
**return** `np.array(dataX), np.array(dataY)`

Below is a basic algorithm that generates one dataset for the purpose of time series prediction based on an input data array and a specified time step length. To begin, it initializes two empty lists called dataX and dataY, which are meant to store input sequences and target values, respectively. For every index 'i' in data up to 'len(data)-time step-1', so that one does not go overboard, it picks, as an array 'a', a sequence of values from current index 'i' to 'i+time step', appending 'a' to 'dataX'. It then picks, as target output, the immediate next value, 'data[i+time step]', and appends that to 'dataY'. This then returns dataX and dataY as numpy arrays that are ready to use for model training.

**Algorithm 2** Prepare Data for Model Training

**Require:** Scaled data `scaled_data`, time step `time_step = 5`
**Ensure:** Training and testing datasets `X_train`, `X_test`, `y_train`, `y_test`
Call `create_dataset(scaled_data, time_step)` to get `X` and `y`
Reshape `X` to have shape $(X.shape[0], X.shape[1], 1)$
Set `train_size` $\leftarrow int(len(X) \times 0.8)$
Split data:
  `X_train` $\leftarrow$ `X[:train_size]`
  `X_test` $\leftarrow$ `X[train_size:]`
  `y_train` $\leftarrow$ `y[:train_size]`
  `y_test` $\leftarrow$ `y[train_size:]`
**return** `X_train, X_test, y_train, y_test`

This algorithm prepares scaled time series data for model training by calling the 'create dataset' function with a specified time step, returning both 'X' (input sequences) and 'y' (target values). It reshapes 'X' to be compatible with sequential models by adding an extra dimension. Then, it calculates the 'train size' as 80% of the total dataset length. It then splits the provided data, according to the 'train size', into 'X' and 'y'. This means that the first 80% of 'X' and 'y' will be attributed to 'X-train' and 'y-train', while the remaining 20% will go to 'X-test' and 'y-test', respectively. It returns a training and a test dataset ready to train and test some model on.

### a) LSTM model

The LSTM architecture is intended to procedure sequences without the need for a large external memory stack, as it keeps activations localized both spatially and temporally. This allows LSTMs to handle distributed data representations efficiently. They are proficient in managing calculations that include continuous values and have the capability to determine and influence the temporal sequence of events, even when these events are separated by long time gaps. This makes LSTMs particularly skilful for tasks where the sequence and timing of data points are critical. [35]

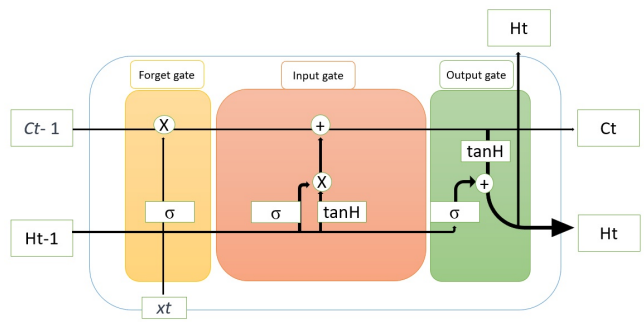


Figure 3. LSTM architecture

The equations governing these components in LSTM unit are as follows:

- Forget Gate: The gatekeeper of relevance, deciding what information to retain or discard from the previous cell state (Ct-1). It employs a sigmoid function ($\sigma$), outputting values between 0 (completely forget) and 1 (completely keep). [36]

$ft = \sigma(Wf\mathring{u}[ht - 1, xt] + bf)$ (1)

- $\sigma$: Sigmoid function for gate activation.
- Wf: Weight matrix for forget gate.
- ht-1: Previous hidden state.
- xt: Current input.
- bf: Bias term for forget gate.

- Input Gate: The selective curator, determining what new information from the current input (xt) is worth incorporating into the cell state. It also employs a sigmoid function ($\sigma$) for decision-making. [36]

$it = \sigma(Wi\mathring{u}[ht - 1, xt] + bi)$ (2)

- $\sigma$: Sigmoid function for gate activation.
- Wi: Weight matrix for input gate.

- ht-1: Previous hidden state.
- xt: Current input.
- bi: Bias term for input gate.

Candidate Cell State: A temporary workspace, where potential additions to the cell state are crafted. It utilizes the tanh function, generating values between -1 and 1, representing both positive and negative contributions.

$Ct = tanh(Wc\mathring{u}[ht-1, xt] + bc)$ (3)

- tanh: Hyperbolic tangent function for non-linearity.
- Wc: Weight matrix for candidate cell state.
- ht-1: Previous hidden state.
- xt: Current input.
- bc: Bias term for candidate cell state.

- Output Gate: The master communicator, deciding what information to release from the cell state as the new hidden state (ht). It employs a sigmoid function ($\sigma$) for deciding what to expose, and then filters the cell state through the tanh function to regulate the output values.[36]

$ot = \sigma(Wo\mathring{u}[ht-1, xt] + bo)$ (4)

$ht = ot * tanh(Ct)$ (5)

- $\sigma$: Sigmoid function for gate activation.
- Wo: Weight matrix for output gate.
- ht-1: Previous hidden state.
- xt: Current input.
- bo: Bias term for output gate.
- Tanh: Hyperbolic tangent function for regulating output values.

**Algorithm 3** Building and Training an LSTM Model

```
Initialize lstm_model as a Sequential model
Add an LSTM layer with 50 units and return_sequences=True, with input shape (time_step, 1)
Add another LSTM layer with 50 units and return_sequences=False
Add a Dense layer with 25 units
Add a final Dense layer with 1 unit
Compile the model with optimizer 'adam', loss 'mean_squared_error', and metric RootMeanSquaredError
Train the model with X_train and y_train for 100 epochs, batch size 1, validation split of 0.2, and callback early_stopping
```

In this model, two dense that are fully connected layers were used in order to transform the final output from the LSTM layers. Here is why two dense layers instead of just one are included in configurations like this:

Feature Transformation and Flexibility: The first dense layer herein acts as an intermediary step to learn a more complex representation of LSTM output with 25 units before being fed to the final dense layer. This layer learns the subtle patterns and relationships that help in enhancing predictive capability. Taking two dense layers in succession lets the model learn a hierarchy of features; for example, the dense layer with 25 units learns intermediate features that the last layer might refine.

Reduction to Target Shape: The second dense layer, with one unit, reduces the model output to target shape. In this case, it's compatible with a target variable, that is one numerical output for regression.

Smoother Optimization and Better Performance: Adding a middle dense layer can sometimes enable the model to generalize better because it is afforded opportunities to learn more intermediate features that may reduce overfitting. Sometimes, an extra dense layer leads to smoother optimization and better performance since there is one more layer of non-linearity added to help capture the complex relationships in the data.

*b) 1D-CNN*

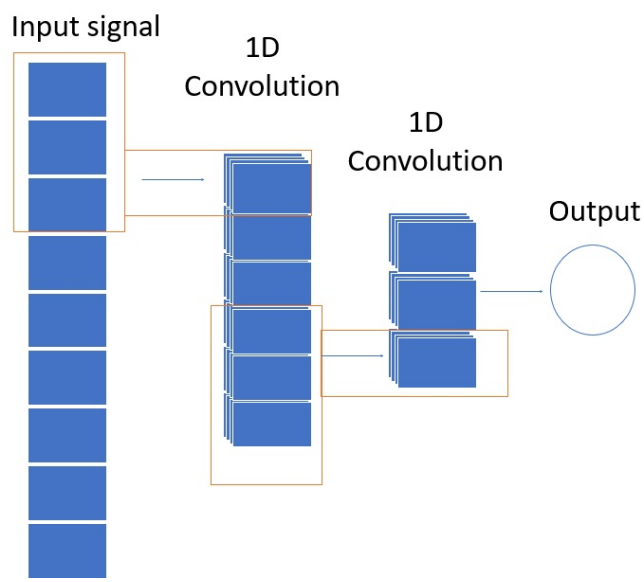


Figure 4. 1D-CNN architecture [37]

1D Convolution: The first layer of 1D convolution involves sliding a filter (or kernel) over the input signal. These contain the vertical red rectangles representing filters. For any position of this filter, its weighted sum with respect to all the encountered values is computed as well as sums form another feature map which is a changed representation

of the initial data. Further 1D Convolutions: The subsequent layers of 1D convolutions take the feature maps produced by the previous layers as their input. Each layer extracts higher-level features. As they go on, convolutions within can detect more complex patterns since they blend features from previous layer's maps. Output: The final stage of the process involves taking the feature maps from last convolutional layers and transforming them into output. This output might be one value (like category in classification), set (such as multiple attributes or scores), or even another sequence.

**Algorithm 4** Building and Training a CNN Model

1: Initialize `cnn_model` as a Sequential model
2: Add a Conv1D layer with 64 filters, kernel size of 2, `relu` activation, and input shape `(time_step, 1)`
3: Add a MaxPooling1D layer with pool size of 2
4: Add a Flatten layer to convert 3D output to 1D
5: Add a Dense layer with 50 units and `relu` activation
6: Add a final Dense layer with 1 unit
7: Compile the model with optimizer `'adam'`, loss `'mean_squared_error'`, and metric `RootMeanSquaredError`
8: Train the model with `X_train` and `y_train` for 100 epochs, batch size 1, validation split of 0.2, and callback `early_stopping`

This CNN model uses two Dense layers for reasons similar to those in the example for the LSTM model. Here's why both layers could be of benefit:

Intermediate Representation with Feature Learning: The model, after convolution and pooling, applies a flattening step, which changes the multi-dimensional data into a one-dimensional vector. First, a dense layer with 50 units is added so that complicated patterns can be learned from the previously flattened vector, further helping to capture the relationships and higher-level features that the convolutional layers have extracted. It allows the model to build up a more complex intermediate representation before the final prediction.

Reduction to Final Output: The second dense layer contains only one unit, mapping the learned features down to desired output size for regression tasks, often to a single unit for the single value prediction of stock price or any other continuous output. This final dense layer ensures that the model output matches the target format.

Non-linear Transformations and Model Capacity: Using two dense layers, along with non-linear activation-relu in this case, would allow the model to capture more of the underlying nonlinear associations among the data. The presence of an intermediate dense layer further increases the capacity for learning complex patterns that may not be possible with just a single dense layer and thereby aids in model accuracy and generalization.

*c) Hyperparameters configurations*

To make the prediction and avoid overfitting, we need to add the EarlyStopping function. (Algorithm 3):
This code sets up an EarlyStopping callback, a technique used to prevent overfitting during training by stopping the model when its performance on the validation set stops improving. Let's go over each of the parameters and why it is useful:

**Algorithm 5** Early Stopping Configuration

1: Initialize `early_stopping` as an EarlyStopping callback
2: Set `monitor` to `'val_loss'` to track validation loss
3: Set `patience` to 10, allowing training to continue for 10 epochs without improvement before stopping
4: Set `min_delta` to $1 \times 10^{-4}$ to ignore improvements smaller than 0.0001
5: Set `mode` to `'min'` to stop training when validation loss stops decreasing
6: Enable `verbose` mode with `verbose=1` to display stopping information
7: Set `restore_best_weights` to `True` to revert to the best weights when stopping is triggered

monitor='val loss': This tells the model to watch the validation loss (val loss) as the metric for improvement. When the validation loss stops decreasing, this callback will consider stopping the training.

patience=10: This is the number of epochs to wait for improvement in val loss before stopping the training. That means here, if the validation loss does not improve in 10 consecutive epochs, then the training stops. A very important reason behind early stopping is to avoid minor loss fluctuations.

min delta=$1e-4$: Minimum improvement in val loss required to be considered. This means that $1e-4$, or 0.0001, is the minimum needed in order to reset the patience counter. Smaller changes are henceforth ignored, which will serve to avoid stopping because of minimal improvements that may not be meaningful.

mode='min': Because the objective here is to minimize the validation loss, mode='min' is one way to instruct the callback to stop when val loss stops decreasing. For metrics where higher is better, such as accuracy, mode='max' would be used instead.

verbose=1: This turns on verbose output, which means the model will print a message when early stopping is triggered. This gives good tracking whenever the training stops and why it has stopped, which is handy for debugging and monitoring purposes.

restore best weights=True:, by default, this flag has the effect that when early stopping is triggered, the weights

used by the model revert to being the best weights seen so far during training-so, the final model is the best-performing version seen, rather than the one from the last epoch.

In a nutshell, this setup of EarlyStopping aims to achieve model performance optimization through stopping the training where further improvements are unlikely, guards against overfitting, and retains the best state of the model.

*d) Evaluation metrics*

This table presents the metrics, namely RMSE and MAE, which are utilized for the computation and assessment of our dataset.

TABLE II. Evaluation metrics.[38]

| Metric | Description | Definition |
|---|---|---|
| MAE | Mean Absolute Error | $\frac{1}{n}\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$ |
| RMSE | Root Mean Squared Error | $\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$ |
| MAPE | Mean Absolute Percentage Error | $\frac{100}{n}\sum_{i=1}^{n}\left\lvert\frac{y_i - \hat{y}_i}{y_i}\right\rvert$ |
| $R^2$ | Coefficient of Determination | $1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$ |

Directional Accuracy :

$$DA = \frac{1}{n-1}\sum_{i=1}^{n-1} \nVdash [\text{sign}(y_{i+1} - y_i) = \text{sign}(\hat{y}_{i+1} - \hat{y}_i)]$$

Where:

- n is the number of observations.
- yi is the actual value for the ith observation.
- ŷi is the prediction for the ith observation.

*e) Wilcoxon Signed-Rank Test*

To assess whether the difference in forecasting performance between the CNN and LSTM models is statistically significant, we employed the Wilcoxon Signed-Rank Test. This non-parametric test is suitable for comparing two related samples, in our case, the absolute prediction errors of both models over the same test dataset.

Let $e_i^{(\text{CNN})}$ and $e_i^{(\text{LSTM})}$ be the absolute errors of the CNN and LSTM models, respectively, for each test observation $i$. We define the difference in errors as:

$$d_i = e_i^{(\text{CNN})} - e_i^{(\text{LSTM})}$$

The Wilcoxon test proceeds by:

- Removing all pairs where $d_i = 0$,
- Ranking the absolute differences $|d_i|$ from smallest to largest, with average ranks assigned in the case of ties,
- Re-assigning the signs of the original differences to their corresponding ranks,
- Computing the test statistic $W$ as the sum of ranks for the positive differences:

$$W = \sum_{i:d_i>0} R_i$$

The null hypothesis ($H_0$) assumes that the median of the paired differences is zero (i.e., both models have equivalent performance). A small p-value ($p \leq 0.05$) leads to rejecting $H_0$, indicating a statistically significant difference in errors.

**Algorithm 6** Wilcoxon Signed-Rank Test for Model Comparison

1: **Input:** Actual values $y_{\text{test}}$, predicted values $\hat{y}_{\text{cnn}}, \hat{y}_{\text{lstm}}$
2: Compute absolute errors:
$e_{\text{cnn}} \leftarrow |y_{\text{test}} - \hat{y}_{\text{cnn}}|$
$e_{\text{lstm}} \leftarrow |y_{\text{test}} - \hat{y}_{\text{lstm}}|$
3: Truncate to common length:
$n \leftarrow \min(\text{len}(e_{\text{cnn}}), \text{len}(e_{\text{lstm}}))$
$e_{\text{cnn}} \leftarrow e_{\text{cnn}}[:n], \quad e_{\text{lstm}} \leftarrow e_{\text{lstm}}[:n]$
4: Perform Wilcoxon Signed-Rank Test:
(statistic, p_value) $\leftarrow$ `wilcoxon`($e_{\text{cnn}}$, $e_{\text{lstm}}$, zero_method='wilcox')
5: **Output:** Test statistic and p-value

We computed the absolute prediction errors of both models on the same test dataset and applied the Wilcoxon test to determine whether the differences in these errors were statistically significant.

Before applying the test, the error vectors were truncated to the same length to ensure proper pairing. The test was conducted using the `scipy.stats.wilcoxon` function in Python, with the parameter `zero_method='wilcox'` to ignore zero differences. The output of the test included the Wilcoxon test statistic and the associated $p$-value, which were used to assess the significance of the observed performance differences between the models.

## 5. Results

### A. Evaluation metrics

In our study, we utilized RMSE, MAE, MAPE, $R^2$ and Direction Accuracy metrics for testing predictions to assess the performance of the models. The results are detailed in the table 3 provided.

*a) Interpretation:*

Comparative analysis of the two models indicates that LSTM is generally more superior in performance, particularly in predictive accuracy and error minimization. With a significantly lower Root Mean Squared Error (RMSE) of 90.97 compared to 102.51 for Model CNN, LSTM demonstrates higher ability in punishing outliers from the true values a highly critical attribute needed for financial forecasting where outlier errors can be greatly damaging.

TABLE III. Comparative Performance Metrics of LSTM and 1D-CNN Models on Price Prediction

| Metric | LSTM | CNN |
|---|---|---|
| RMSE | **90.9658** | 102.5054 |
| MAE | **65.7296** | 72.1576 |
| MAPE | **0.006053** | 0.006644 |
| $R^2$ | **0.974356** | 0.967437 |
| Direction Accuracy | 0.525862 | **0.538793** |

Similarly, LSTM Mean Absolute Error is 65.73, which is much less than that of CNN at 72.16, confirming its reliable ability to generate more accurate point forecasts on the data set. The Mean Absolute Percentage Error (MAPE), which reflects relative error and is especially very important in heterogeneous financial time series, is also lower for LSTM (0.00605 vs 0.00664), confirming its greater precision in capturing percentage-based fluctuations in stock prices.

The R-squared ($R^2$) metric also validates LSTM's dominance, as it achieves a value of 0.9744, which is higher than that of CNN at 0.9674. What this indicates is that LSTM accounts for a greater proportion of the variance in the target variable, meaning that it has a tighter model fit with better generalization performance on unseen data. Such a high $R^2$ for stock market prediction indicates that the model can effectively capture non-linear complex financial dynamics.

But when looking at Directional Accuracy a measure that focuses on how well the model can accurately forecast the direction of price changes, not their absolute values CNN is very marginally better than LSTM (0.5388 compared to 0.5259). This implies that while LSTM will be overall more accurate at predicting the exact numerical values, CNN may have a slight edge in identifying trends in the direction of increase or decrease, and this can prove highly important for some trading methods based more on trend than value accuracy.

In conclusion, although CNN only slightly improves directional sensitivity, LSTM clearly offers a better balanced and effective approach to financial times series forecasting. Its lesser margins of error and higher explanatory ability justify it as a more robust and reliable model, especially in processes where accuracy and generalisability are more important than short-term trend identification.

*b) Statistical Comparison using Wilcoxon Signed-Rank Test*

In our experiment, the test yielded:

$$W = 11039, \quad p = 0.01187$$

Since $p < 0.05$, we reject the null hypothesis and conclude that the LSTM model performs significantly better than the CNN model in terms of absolute forecasting error.

### *B. Visualisations*

Figure 5, is a visual representation of the performance of LSTM models, with 80% for the training data presented by the red line and 20% for the testing data represented by green line, the change from training part to testing part in the figure does not provide a clearly decline in performance for the model which means strong generalization capabilities, the plot shows a better performance by the lSTM, which closely follows the original data line and accurately predicts the actual price movements.

Figure 6 depicts some deviations in the CNNs' predictions for unseen data in both phases, while the LSTM can closely follow the actual data. This in turn points to good adaptability probably because of its ability to remember and understand long-term dependencies in sequences of data. Overfitting cannot explain the presence of unseen data since we followed early stopping mechanisms while training the networks. This approach has been very effective in making the learning process optimal without allowing the model to over-fit to the data on which the training is conducted.

Figure 7 provides both CNN and LSTM predictions fit the training data well, with very similarly matched curves. The LSTM model is slightly better and more stable and smoother. On test performance, the LSTM model is considerably more stable, with less variance and smoother predictions. The CNN model, meanwhile, is more erratic in its behavior, with large under and over shooting. For trend capture, the CNN can capture some general trends but with less accuracy compared to the LSTM, where the latter is showing stronger generalization of the price trends. Overall, the LSTM model is better than the CNN in terms of more accurate and reliable prediction.

Figure 8 shows the learning curves for the LSTM model, where the training and validation losses decrease smoothly and steadily over epochs. The close tracking of the two curves is the signature of good learning and good generalization performance, and there is little evidence of overfitting. For comparison, Figure 9 illustrates the training dynamics of the 1D-CNN model, where although the training loss is low and stable, the validation loss has large variability, with steep spikes at a number of epochs. Such deviations are reflective of instability in the CNN model's generalization performance and are characteristic of a tendency to overfit. Overall, while both models exhibit low training losses, the LSTM model is considerably better than the CNN model in validation stability and generalization to new unseen data.

Figure 10 graphs the histogram of prediction residuals for the LSTM and 1D-CNN models. Both histograms are centered around zero, indicating that the models are, on average, unbiased in their predictions. However, closer inspection reveals significant differences in the spread and symmetry of the residuals.

The LSTM residual histogram is narrower and more

symmetric, with most errors concentrated between -100 and +100. The shape is near a normal distribution, which in regression problems is welcome since it shows that the errors in predictions are random and not systematically biased. The lack of heavy tails is also an indicator of the absence of frequent large errors.

In contrast, the CNN residual histogram is more dispersed and has a slightly skewed distribution with more frequent larger positive errors. The distribution is less concentrated and has greater kurtosis, which indicates that the CNN model is experiencing more outliers as well as greater variance in its prediction errors compared to the LSTM.

Figure 11 presents actual and predicted price plots for the most recent 100 time steps for the LSTM and CNN models. The plots allow for the direct visual comparison of how well each model tracks the actual market trend.

In the LSTM plot, the forecasted values follow the actual prices with minimal lag and great precision. The model learns the general rising and falling trends successfully, along with the local fluctuations. This close agreement in all epochs is the evidence that the LSTM model generalizes well and maintains temporal coherency in its forecasts.

In contrast, the CNN predicted curve, although following the overall trend in general, exhibits more exaggerated deviations, especially around peaks and inflection points. Some over and under shooting phenomena are observed, where the predicted curve leads or lags behind actual price trends. This indicates lower sensitivity to dynamic movement, and a lower capability to capture complex temporal dependencies compared to LSTM.

Figures 12 and 13 graph the scatter plots of predicted vs actual prices on the test dataset for the LSTM and CNN models, respectively. Each point is a test observation, and the ideal situation is when all points lie along the 45-degree diagonal line, indicating perfect prediction.

In Figure 12, the LSTM model displays close alignment along the diagonal, with most points closely clustered around the perfect line. This suggests that the LSTM model is highly accurate in its prediction of price values, with consistency throughout the range of outputs. The tight clustering of points reflects low residual error and good generalization to new data.

Conversely, Figure 13 shows the results for the CNN model, where although the general trend still follows the diagonal line, the dispersion of the points is slightly broader. This illustrates greater deviation from predicted to actual values, showing less precise predictions compared to the LSTM model. Under and overestimation by the CNN model more regularly is observed, supporting earlier evidence in the loss and residual plots.

Figures 14 and 15 display the residual plots of the LSTM and 1D-CNN models, respectively, plotting the prediction error over time. The LSTM model residuals (Figure 14) are relatively centered around zero with moderate variance and without any visible systematic pattern. This behavior indicates that the LSTM model's prediction errors are randomly distributed, a desirable property suggesting that the model has successfully captured the underlying structure of the time series without leaving significant patterns in the residuals.

In comparison, the CNN residuals (Figure 15) exhibit a greater amplitude of fluctuation, with additional large spikes and more dispersed spread along the zero line. This suggests higher variance and less stable prediction behavior, which implies that the CNN model has not succeeded in capturing the temporal dynamics as successfully as the LSTM model. The presence of numerous abrupt deviations also suggests poorer error consistency, worse generalization.

As can be observed from the analysis of Figures 5 through 15, the LSTM model is better than the 1D-CNN model in terms of predicting stock price movements across multiple evaluation perspectives, including trend tracking, generalization, residual analysis, and prediction stability. The LSTM generalizes better to new, unseen data, with stable and smooth predictions and low variance, following actual price movements closely, and effectively modeling long-range temporal dependencies without overfitting a fact reinforced by tightly coupled train and validation loss curves. Residual and histogram analysis also provide additional evidence towards its validity, with errors concentrated near zero, thinner distributions, and near-normal shapes, indicating random and unbiased deviations. On the other hand, the CNN model has more fluctuating validation losses, less stable residual patterns with greater amplitude fluctuations, greater spread in predicted vs actual plots, and more frequent large prediction errors, all of which are signs of worse temporal modeling and greater sensitivity to noise. Overall, the LSTM exhibits stronger predictive power, better generalization to new data, and more robust error behavior, and is a more stable choice for modeling complex stock market trends than the CNN in our case.

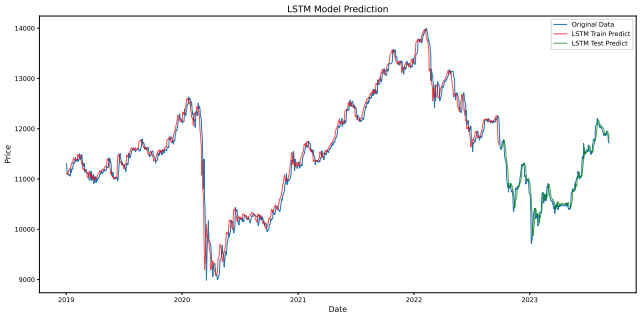


Figure 5. LSTM model prediction

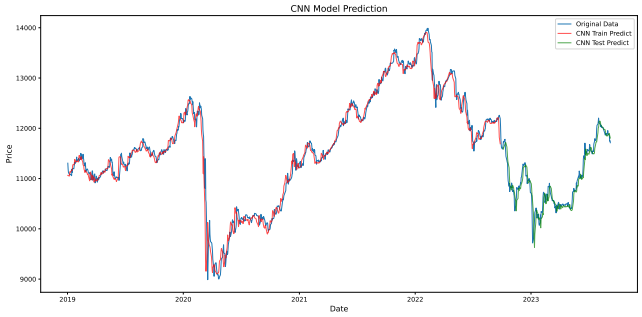


Figure 6. 1D-CNN model prediction

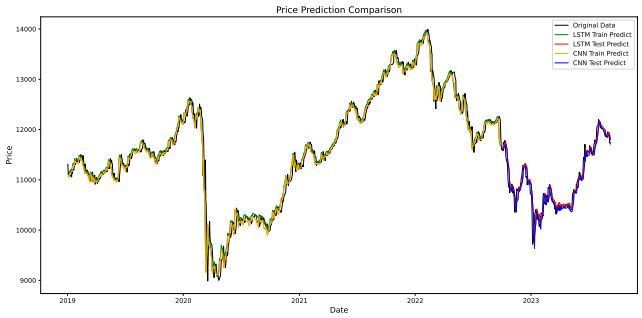


Figure 7. Comparative Analysis of LSTM and CNN Model Performance in Price Prediction

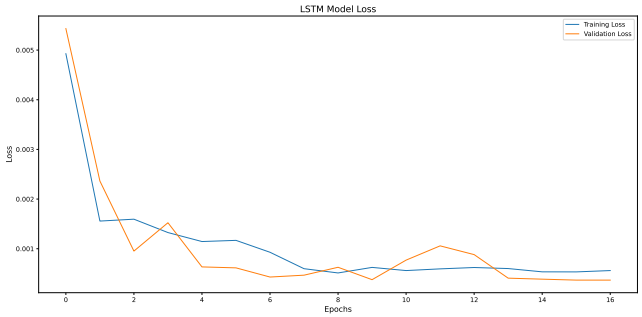


Figure 8. LSTM model loss

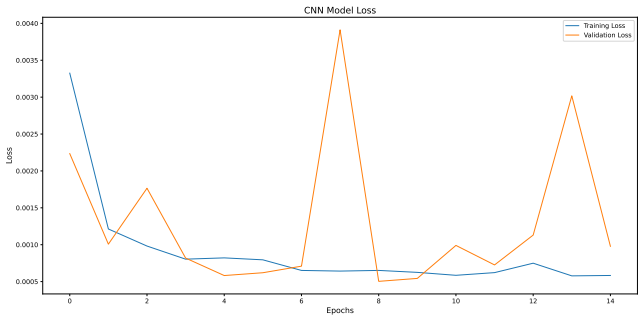


Figure 9. 1D-CNN model loss

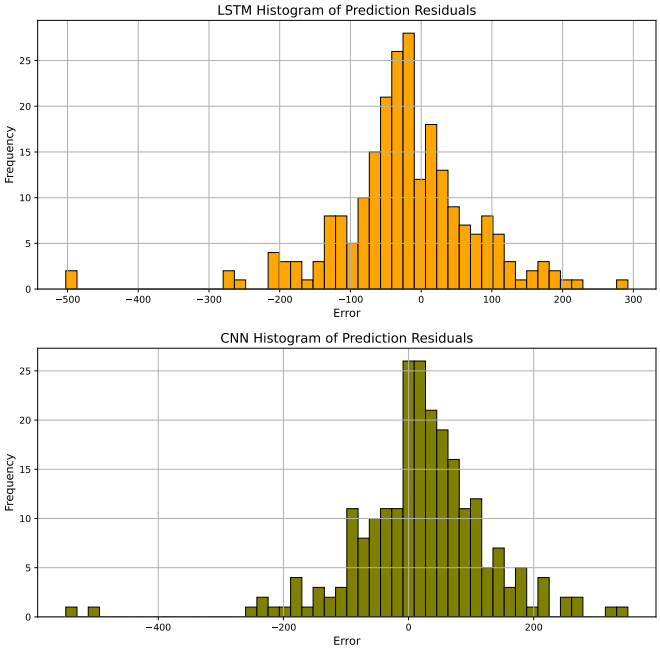


Figure 10. Histogram of prediction residuals

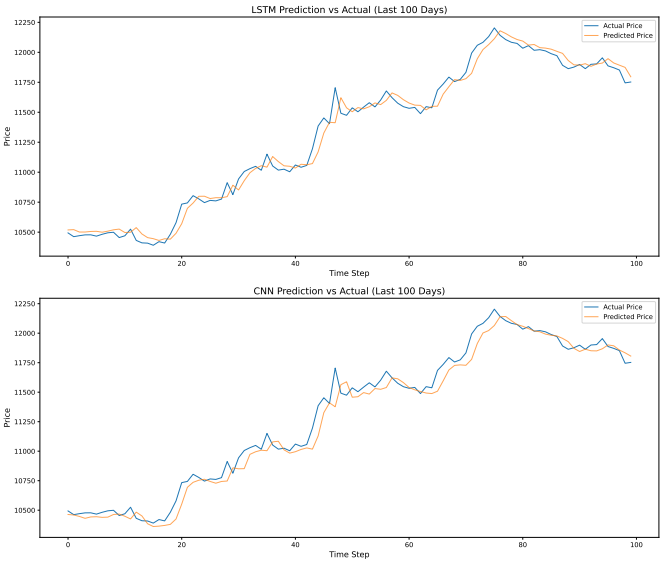


Figure 11. 100 days test prediction

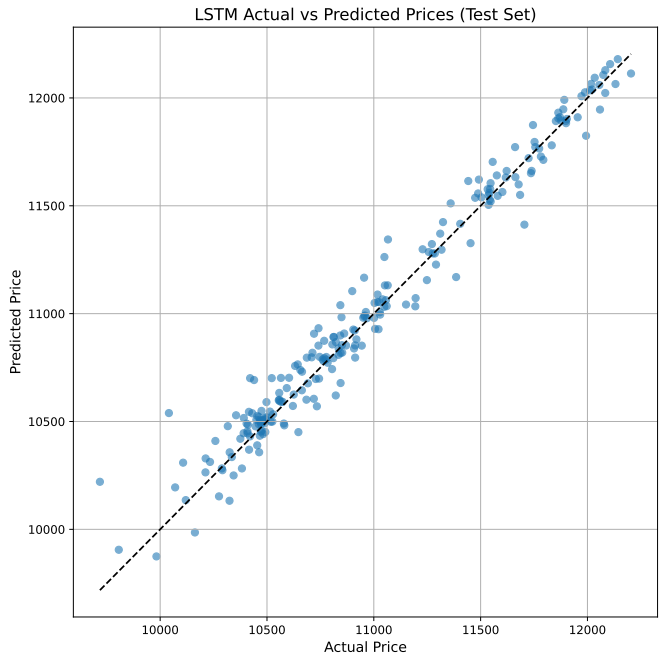


Figure 12. LSTM Actual vs Predicted Test set

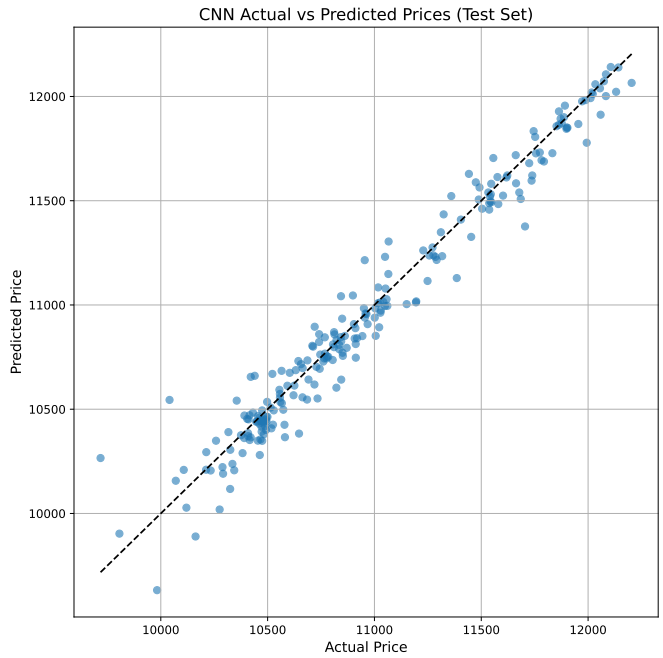


Figure 13. CNN Actual vs Predicted Test set

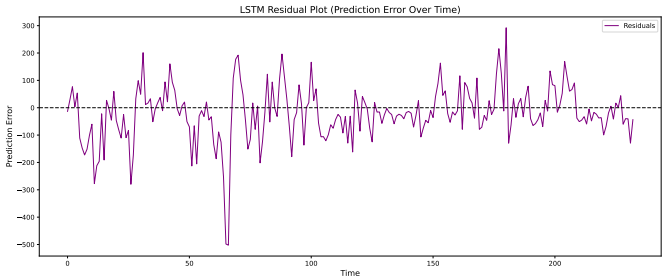


Figure 14. LSTM residual

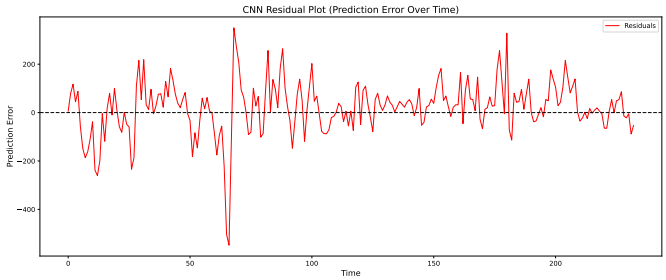


Figure 15. 1D-CNN residual

## 6. Conclusion and Future Work

Each of the models was closely following the trend in actual data during training but with slight deviations; this deviation was more evident in the CNN when it came to testing. These disparities within CNN predictions for unseen data reveal some weaknesses in generalizing capabilities even after being trained properly. However, such a deviation is not because of overfitting since precautions like early stopping have been taken. This helped stop training before models could overfit themselves on data hence making sure learning remained optimal and generalized better with new inputs.

What caught much attention during testing was how well LSTM performed which mirrored actual data thus proving its reliability as well as robustness for practical applications that rely heavily on accurate prediction. Conversely, CNN showed strong capabilities although this might need some further adjustments so as to work better with unknown information by probably changing its response towards fluctuations in numbers.

In our future work, we envisage a hybrid model, combing the statistical power of ARIMA with deep learning capabilities of LSTM. The promising combination is because the ARIMA model is good at modeling the linear relationship while LSTM is very effective at detecting nonlinear patterns. These models, when integrated, will help us explore both approaches for their optimal contributions to our model performance, thus probably leading to the development of a stronger model which can capture both linear and nonlinear dynamics in data. We believe this hybrid framework could further improve forecasting performance and have wider applications for comprehensive analytics of multivariate time series data in finance.

## Declarations

*Funding*

This study was no funding

*Conflicts of interest/Competing interests*

Authors declare that they have no conflict of interest

## References

[1] A. Ibrahim, "Understanding bitcoin market mechanics using feature engineering, data modeling, and forecasting methods," Ph.D. dissertation, University of Westminster, 2024.

[2] H. A. Selmy, H. K. Mohamed, and W. Medhat, "Big data analytics deep learning techniques and applications: A survey," *Information systems*, vol. 120, p. 102318, 2024.

[3] R. Hyndman, *Forecasting: principles and practice*. OTexts, 2018.

[4] F. M. Shiri, T. Perumal, N. Mustapha, and R. Mohamed, "A comprehensive overview and comparative analysis on deep learning models: Cnn, rnn, lstm, gru," 2023.

[5] J. Maqbool, P. Aggarwal, R. Kaur, A. Mittal, and I. A. Ganaie, "Stock prediction by integrating sentiment scores of financial news and mlp-regressor: A machine learning approach," *Procedia Computer Science*, vol. 218, pp. 1067–1078, 2023.

[6] D. Mishra and M. Posts, "Applications of recurrent neural networks (rnns)," *applications-of-rnn*, 2023.

[7] M. M. Elgammal, F. E. Ahmed, and D. G. McMillan, "The predictive ability of stock market factors," *Studies in Economics and Finance*, vol. 39, no. 1, pp. 111–124, 2022.

[8] A. Kordestani, N. Pashkevich, P. Oghazi, M. Sahamkhadam, and V. Sohrabpour, "Effects of the covid-19 pandemic on stock price performance of blockchain-based companies," *Economic research-Ekonomska istraživanja*, vol. 35, no. 1, pp. 3206–3224, 2022.

[9] R. Asalatha, "Stock markets: An overview and a literature review," *MPRA, Paper*, vol. 101855, 2019.

[10] S. Lu, C. Liu, and Z. Chen, "Predicting stock market crisis via market indicators and mixed frequency investor sentiments," *Expert Systems with Applications*, vol. 186, p. 115844, 2021.

[11] A. T. Oyewole, O. B. Adeoye, W. A. Addy, C. C. Okoye, O. C. Ofodile, and C. E. Ugochukwu, "Predicting stock market movements using neural networks: a review and application study," *Computer Science & IT Research Journal*, vol. 5, no. 3, pp. 651–670, 2024.

[12] E. F. Fama, "Efficient capital markets," *Journal of finance*, vol. 25, no. 2, pp. 383–417, 1970.

[13] Autorité Marocaine du Marché des Capitaux, "Moroccan market capital authority: Market reports," 2024, accessed: 2024-11-12.

[14] Casablanca Stock Exchange, "Casablanca stock exchange official website," 2024, accessed: 2024-11-12.

[15] Bank Al-Maghrib, "The casablanca stock exchange: Financial markets infrastructure and monitoring," 2024, accessed: 2024-11-12.

[16] J. L. Elman, "Finding structure in time," *Cognitive science*, vol. 14, no. 2, pp. 179–211, 1990.

[17] T. Phuoc, P. T. K. Anh, P. H. Tam, and C. V. Nguyen, "Applying machine learning algorithms to predict the stock price trend in the stock market–the case of vietnam," *Humanities and Social Sciences Communications*, vol. 11, no. 1, pp. 1–18, 2024.

[18] S. Hochreiter, "Long short-term memory," *Neural Computation MIT-Press*, 1997.

[19] F. Ferdiansyah, S. H. Othman, R. Z. M. Radzi, D. Stiawan, and T. Sutikno, "Hybrid gated recurrent unit bidirectional-long short-term memory model to improve cryptocurrency prediction accuracy," *IAES International Journal of Artificial Intelligence*, vol. 12, no. 1, p. 251, 2023.

[20] F. Ricchiuti and G. Sperlí, "An advisor neural network framework using lstm-based informative stock analysis," *Expert Systems with Applications*, vol. 259, p. 125299, 2025.

[21] B. Gülmez, "A hybrid approach for stock market price forecasting using long short-term memory and seahorse optimization algorithm," *Annals of Data Science*, pp. 1–27, 2025.

[22] S. Agarwal, S. Sharma, K. N. Faisal, and R. R. Sharma, "Time-series forecasting using svmd-lstm: A hybrid approach for stock market prediction," *Journal of Probability and Statistics*, vol. 2025, no. 1, p. 9464938, 2025.

[23] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[24] S. A. Farimani, M. V. Jahan, and A. M. Fard, "An adaptive multimodal learning model for financial market price prediction," *IEEE Access*, 2024.

[25] J. Natarajan, W. Wang, Y. Jiang, Z. Zhang, H. Ye, and L. Kuang, "Generative-cnn for pattern recognition in finance," in *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024, pp. 142–149.

[26] P. P. Monfared and S. M. Jarchi, "Cnn-nlp based persian feedback detection of stock symbols in the stock market," in *2024 10th International Conference on Artificial Intelligence and Robotics (QICAR)*. IEEE, 2024, pp. 125–130.

[27] W. Lu, J. Li, Y. Li, A. Sun, and J. Wang, "A cnn-lstm-based model to forecast stock prices," *Complexity*, vol. 2020, no. 1, p. 6622927, 2020.

[28] J. M.-T. Wu, Z. Li, N. Herencsar, B. Vo, and J. C.-W. Lin, "A graph-based cnn-lstm stock price prediction algorithm with leading indicators," *Multimedia Systems*, vol. 29, no. 3, pp. 1751–1770, 2023.

[29] X. Zhang, "Analyzing financial market trends in cryptocurrency and stock prices using cnn-lstm models," 2024.

[30] B. H. A. Khattak, I. Shafi, C. H. Rashid, M. Safran, S. Alfarhood, and I. Ashraf, "Profitability trend prediction in crypto financial markets using fibonacci technical indicator and hybrid cnn model," *Journal of Big Data*, vol. 11, no. 1, p. 58, 2024.

[31] A. K. Yadav and V. P. Vishwakarma, "An integrated blockchain based real time stock price prediction model by cnn, bi lstm and am," *Procedia Computer Science*, vol. 235, pp. 2630–2640, 2024.

[32] F. Li, H. Zhou, M. Liu, and L. Ding, "A medium to long-term multi-influencing factor copper price prediction method based on cnn-lstm," *IEEE Access*, vol. 11, pp. 69 458–69 473, 2023.

[33] Z. Dong and Y. Zhou, "A novel hybrid model for financial forecasting based on ceemdan-se and arima-cnn-lstm," *Mathematics*, vol. 12, no. 16, p. 2434, 2024.

[34] B. P. Ghosh, M. S. Bhuiyan, D. Das, T. N. Nguyen, R. M. Jewel, M. T. Mia, D. M. Cao, and R. Shahid, "Deep learning in stock market forecasting: Comparative analysis of neural network architectures across nse and nyse," *Journal of Computer Science and Technology Studies*, vol. 6, no. 1, pp. 68–75, 2024.

[35] K. Gajamannage, Y. Park, and D. I. Jayathilake, "Real-time forecasting of time series in financial markets using sequentially trained dual-lstms," *Expert Systems with Applications*, vol. 223, p. 119879, 2023.

[36] Y. Z. JING and D. PATHMANATHAN, "Financial market predictions with deep learning," *Journal of Quality Measurement and Analysis JQMA*, vol. 19, no. 2, pp. 81–97, 2023.

[37] A. Shenfield and M. Howarth, "A novel deep learning model for the detection and identification of rolling element-bearing faults," *Sensors*, vol. 20, no. 18, p. 5112, 2020.

[38] T. O. Hodson, "Root mean square error (rmse) or mean absolute error (mae): When to use them or not," *Geoscientific Model Development Discussions*, vol. 2022, pp. 1–10, 2022.